VIVEKANAND EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

MICRO PROJECT REPORT

Project Title : Fault-Tolerant Distributed Lock Service (Raft consensus, leases and fencing tokens)

SDG Aligned & Logo : SDG 9 (Industry, Innovation and Infrastructure); SDG 4 (Quality Education)





Project / Course Details	To be filled by students
Course / Subject Name	Software System Design
Course Code / Semester / Academic Year	26CMPE52 / V / 2026-27
Faculty Incharge / Project Mentor	Mrs. Priya R.L.
Project Group No. (if applicable)	13
Submission Date	

Project Team Details

Student Name	Roll No.	Class / Division	Institute Email ID
Tanmay Maniyar	33	D12A	2024.tanmay.maniyar@ves.ac.in

Department of Computer Engineering

Academic Year: 2026-27

Table of Contents

Abstract 4

1. Introduction 4

1.1 Background and Motivation 4

1.2 Problem Context 4

1.3 Intended Users / Stakeholders 5

2. Problem Definition, Objectives and Scope 5

2.1 Problem Statement 5

2.2 Objectives 5

2.3 Scope and Deliverables 6

2.4 Assumptions and Constraints 6

3. Background / Literature Review 6

3.1 Identified Gap and Project Contribution 7

4. Proposed Solution and System Design 7

4.1 Proposed Solution 7

4.2 System Architecture 7

4.3 Module Description 9

4.4 Data / Inputs / Dataset 9

4.5 Algorithm / Design Decisions 10

5. Methodology and Implementation 13

5.1 Workflow / Development Method 13

5.2 Technologies, Tools and Resources 13

5.3 Implementation Details 13

5.4 User Interface / Prototype / Output 14

5.5 Team Contribution 21

6. Testing, Evaluation and Results 21

6.1 Test Plan and Test Cases 21

6.2 Evaluation Metrics 23

6.3 Results and Discussion 24

6.4 Validation / Demonstration Evidence 24

7. Sustainability and SDG Alignment 25

7.1 Sustainability Assessment 25

7.2 UN Sustainable Development Goals (SDGs) 25

7.3 Sustainability Measures and Trade-offs 25

7.4 Sustainability Indicator / Follow-up 26

8. Feasibility, Limitations and Risk Considerations 26

8.1 Feasibility 26

8.2 Limitations 26

8.3 Risks, Ethics, Privacy and Security 26

9. Conclusion and Future Scope 27
9.1 Summary 27
9.2 Future Scope 27
References 28
Appendices 29
Appendix A: Additional screenshots / outputs 29
Appendix B: Selected pseudocode or code excerpts 31
Appendix C: Additional test cases 32
Appendix D: Repository / demo / deployment information 32

Note: if you edit the report, right-click the table of contents in Word and choose "Update Field" to refresh the page numbers.

Abstract

Programs that share a resource must take turns, and a lock service is the component that decides whose turn it is. A single lock server is itself a point of failure: if it crashes nobody can work, and if it errs two programs can enter the critical section together. This micro project designs, builds and evaluates a fault-tolerant distributed lock service in the style of Google Chubby and etcd. Five replica processes written in Node.js with no external libraries keep an identical log of lock operations using the Raft consensus algorithm: leader election with pre-vote, log replication, majority commit, check-quorum and a write-ahead log on disk. On top of the log sits a deterministic lock state machine that offers leases (locks that expire on their own) and strictly increasing fencing tokens, so that a client that was paused past its lease cannot damage the protected resource. A gateway finds the leader, retries safe operations, injects crashes, restarts and network partitions, and serves a browser dashboard with a live terminal that streams the real log lines of every process. The system was evaluated with 12 scripted fault-injection checks, a randomised 20-second chaos test that verified mutual exclusion over 116 grants, and benchmarks that measured about 31 ms per operation, up to about 4,300 committed operations per second and a failover time of roughly 0.8 to 1.1 seconds. All 12 checks and all 8 chaos checks passed in the runs reported here. A static Virtual Lab with six experiments, whose cluster is simulated in the browser, supports teaching. The project supports SDG 9 (resilient digital infrastructure) and SDG 4 (quality education).

Keywords: distributed systems, Raft consensus, distributed lock, lease, fencing token, fault tolerance, network partition.

1. Introduction

1.1 Background and Motivation

Modern applications run as many cooperating processes on many machines. Whenever two of them may change the same record, file or device, they need a way to take turns. A **lock** gives one process at a time the right to use the shared thing; this property is called **mutual exclusion**. On one computer the operating system provides locks. Across machines a separate service must provide them, and that service must keep working when machines crash or the network breaks.

This project applies core distributed-systems ideas to that problem: replication, consensus, leader election, quorums, failure detection with timeouts, durability through write-ahead logging, and the safe use of time through leases [2], [6]. It follows the design of production coordination services such as Chubby [2], ZooKeeper [4] and etcd [8], built on the Raft consensus algorithm [1].

1.2 Problem Context

The simplest design is one lock server. It is easy to build but it fails badly: when it crashes every client is blocked, and if it is restarted from stale data it may grant a lock that is already held. Copying the server to several machines creates a new problem, because the copies can disagree, and during a network split two of them may each believe they are in charge (**split brain**) and grant the same lock to different clients.

A second, subtler problem exists even with a correct lock service. A client can be paused for a long time (for example by a garbage-collection pause or a network stall) after it checked that it holds the lock. Its lease then expires, another client takes the lock, and when the first client wakes up it still writes to the shared resource [3]. The gap this project addresses is therefore a lock service that stays correct under crashes, partitions and slow clients, and that can be shown working, step by step, to a student or an examiner.

1.3 Intended Users / Stakeholders

- **Application developers** who need mutual exclusion or leader election between their own processes.
- **Operators** who must see which machine leads, what failed and how the cluster healed (the dashboard and the live terminal serve this need).
- **Students and instructors** of distributed systems, who use the guided dashboard and the Virtual Lab to see Raft behave.
- **Course evaluators**, who need evidence that each claim (safety, recovery, performance) is backed by a test or a measurement.

2. Problem Definition, Objectives and Scope

2.1 Problem Statement

Design and implement a lock service for client programs that (a) never grants the same lock to two clients at the same time, (b) keeps granting and releasing locks while up to two of five machines crash or while the network splits, (c) frees locks held by crashed clients automatically, and (d) gives resources a way to reject writes from clients that no longer hold the lock. The result must be observable (a dashboard and logs) and testable (automated fault-injection checks).

2.2 Objectives

Table 1: Project objectives and how each is verified

ID	Objective (measurable)	How it is verified
O1	Design a 5-replica cluster that agrees on every lock operation through Raft.	Commit lines "stored on 3/5 nodes" in the live terminal; test TC-01 to TC-03
O2	Implement leases and strictly increasing fencing tokens.	TC-07, TC-08; fencing demo; chaos check "tokens unique"
O3	Elect a new leader automatically after a crash and keep committed locks.	TC-04 to TC-06; failover benchmark
O4	Remain safe under network partitions and heal without split brain.	TC-09 to TC-12; partition screenshots
O5	Persist term, vote and log so a restarted node recovers its state.	Disk lines in the terminal; chaos check "logs identical"
O6	Evaluate latency, throughput and failover time, and test safety under random faults.	docs/BENCHMARKS.md; chaos test
O7	Provide a dashboard with a live backend terminal and a Virtual Lab with six experiments.	Screenshots in Section 5.4

2.3 Scope and Deliverables

Included: the replica (Raft and lock state machine), the gateway, the dashboard, fault injection, the fenced resource, tests, a chaos test, benchmarks, deployment notes (Docker Compose and a multi-host option), and the Virtual Lab (a separate static website).

Excluded: log compaction and snapshots, changing the cluster membership while running, Byzantine (malicious) faults, TLS and user accounts, and a validated cloud deployment. These are listed again as limitations and future scope.

Deliverables: the working prototype (`mini-project/`), the Virtual Lab (`vlab/`), the test and benchmark scripts, the written design documents, a demo script, and this report.

2.4 Assumptions and Constraints

- **Fault model:** machines fail by crashing and may recover; messages may be lost or delayed but are not forged (crash-fault model, not Byzantine).
- **Clocks:** lease times are stamped by the leader into each command, so all replicas apply the same decision; the design assumes the leader's clock is reasonably accurate compared with the lease length.
- **Environment:** all five replicas run as local processes on one laptop and talk over loopback HTTP; partitions and crashes are injected in software. The code can be pointed at several hosts, but that was not tested for this report.
- **Constraint:** no third-party libraries; Node.js 18 or newer is the only requirement.
- **Privacy:** the system stores only lock names and client names that the user types; it holds no personal data.

3. Background / Literature Review

The design draws on the sources below. Each row says what the source contributes and how it shaped the project.

Table 2: Review of related work

Source / Year	Approach or Key Finding	Strength / Limitation	Relevance to This Project
Ongaro and Ousterhout, "In Search of an Understandable Consensus Algorithm", 2014 [1]	Raft: leader election, log replication and safety rules, split into small understandable parts.	Clear and complete; the paper leaves practical details (storage, client interface) to the implementer.	Basis of `replica.js`: terms, votes, AppendEntries, majority commit.
Burrows, "The Chubby Lock Service", 2006 [2]	Coarse-grained locks and small files on a 5-replica cell used by Google.	Proven in production; closed source.	Model for the service: 5 replicas, leases, a lock service used for coordination.
Hunt et al., "ZooKeeper", 2010 [4]	Wait-free coordination service with ordered updates and ephemeral nodes.	Very general; a heavy dependency for a small system.	Comparison point; shows why a focused lock API is simpler.
Kleppmann, "How to do distributed locking", 2016 [3]	A lease alone is unsafe because a paused client does not know it expired; use fencing tokens checked by the resource.	Practical argument; needs a resource that can check tokens.	Source of the fencing-token design and the stale-write demo.
Gray and Cheriton, "Leases", 1989 [6]	Time-limited grants let a server recover from failed clients without waiting forever.	Assumes bounded clock drift.	Locks are leases that expire and are swept through the log.

Source / Year	Approach or Key Finding	Strength / Limitation	Relevance to This Project
Ongaro, PhD dissertation, 2014 [5]	Extends Raft with refinements such as Pre-Vote and leaders stepping down when they lose their majority.	Detailed; some refinements add code.	Pre-vote and check-quorum are implemented and can be switched on and off.
etcd documentation [8]	A production key-value store that uses Raft for leases and locks.	Mature; hides most internals.	Reference for expected behaviour of leases and leader failover.

3.1 Identified Gap and Project Contribution

Production systems [2], [4], [8] are reliable but are large, and their internal behaviour is hidden from a learner. Teaching implementations of Raft often stop at election and replication and do not connect them to a lock service with leases and fencing. This project contributes a small, dependency-free, complete path from a click on a button to a committed lock, with every step visible in a live log, plus a way to break the system on purpose and a check that proves it stayed safe. This is an implementation and demonstration contribution, not a new algorithm.

4. Proposed Solution and System Design

4.1 Proposed Solution

The solution has three layers. **Replicas** (`replica.js`, five processes) run Raft and a lock state machine. The **gateway** (`server.js`) is the front door: it finds the leader, retries safely, starts and stops the replicas and injects faults. The **dashboard** (`public/`) is a single web page that polls the gateway every 500 ms and draws the cluster, the locks and the live terminal.

Workflow of a user: open the dashboard, look at the ring of five machines, acquire a lock as `client-A`, ask for it again as `client-B` and be refused, crash the leader and watch a new one take over with the lock intact, cut the network and watch only the majority side keep working, and finally run the fencing demo in which a stale write is rejected.

4.2 System Architecture

Figure 1 shows the components. The browser talks only to the gateway. The gateway talks to the replicas over HTTP on `127.0.0.1:7001` to `7005`. The replicas talk to each other with three Raft messages (`/raft/prevote`, `/raft/vote`, `/raft/append`). Each replica owns two files on disk, a write-ahead log (`nodeN.wal`) and a small term-and-vote file (`nodeN.json`), written before the replica replies.

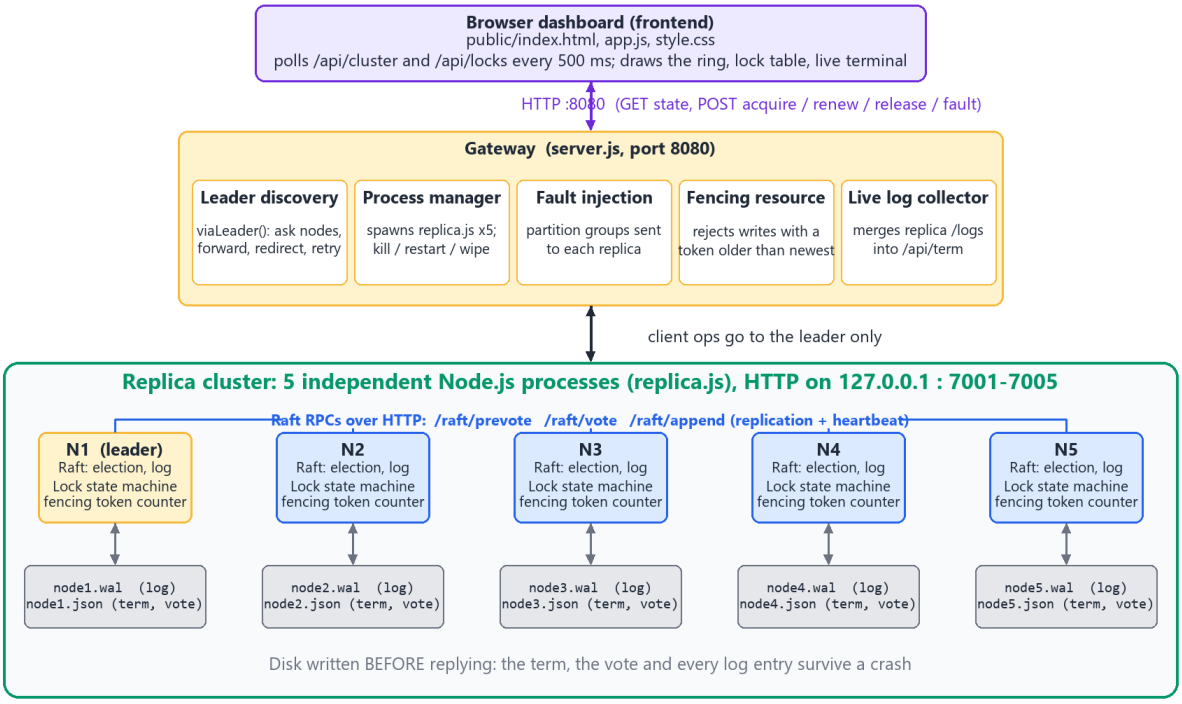


Figure 1: System architecture: browser, gateway and the five replica processes with their disk files

Every node is in one of three states (Figure 2). A node starts as a follower. If it hears no heartbeat for a random 600 to 1100 ms it becomes a candidate and asks for votes. With 3 of 5 votes (a majority) it becomes the leader. A leader that sees a higher term, or that cannot reach a majority for 1.5 s, steps down.

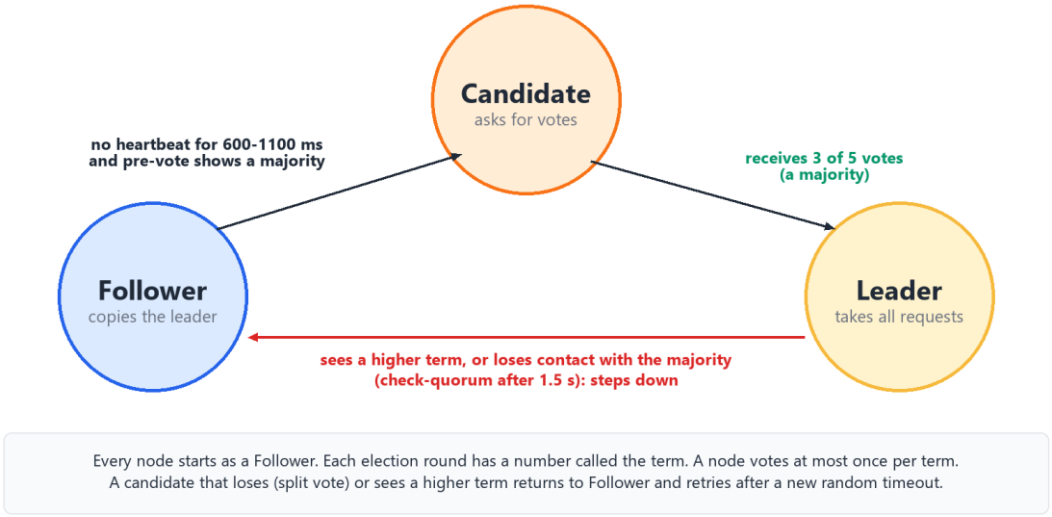


Figure 2: Raft node states and the events that move a node between them

Figure 3 follows one Acquire request. The leader writes the entry to its own disk, sends it to the four followers, and each follower saves it and acknowledges. When the leader has the entry on 3 of 5 machines the entry is **committed**; only then is the lock granted and the client answered. Any later leader is elected by at least 3 machines, and any 3 machines overlap with the 3 that stored the entry, so a committed lock is never lost.

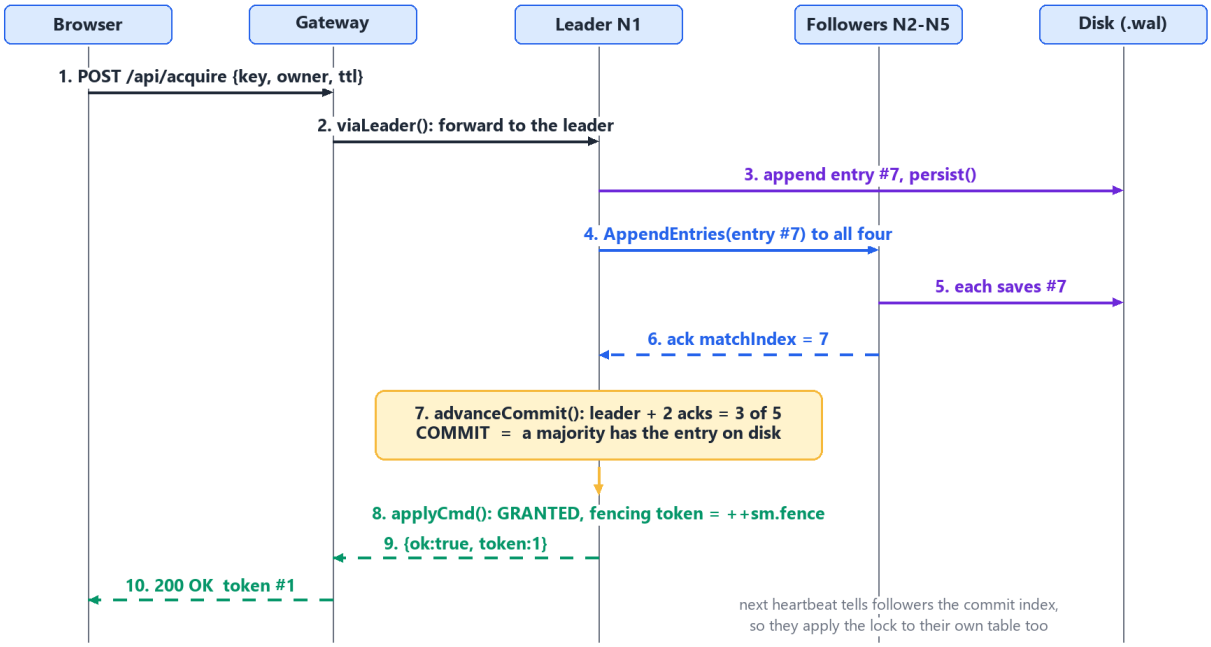


Figure 3: Life of one Acquire request: append, replicate, commit, apply, reply

4.3 Module Description

Table 3: Modules and their responsibilities

Module / Component	Responsibility	Input	Output
Replica: Raft (`replica.js`)	Election with pre-vote, log replication, majority commit, check-quorum, step-down.	Raft RPCs from peers; client commands from the gateway	Replicated, committed log; role and term
Lock state machine (`applyCmd`)	Applies acquire, renew, release and sweep deterministically; issues fencing tokens.	Committed log entries	Grant, refusal or expiry; token number
Persistence (`persist`, `loadDisk`)	Writes term, vote and log before replying; rebuilds state after a crash.	State changes	`nodeN.wal`, `nodeN.json`; recovered state
Gateway (`server.js`)	Finds the leader, forwards and retries, spawns and kills replicas, injects partitions, hosts the fenced resource, merges logs.	HTTP calls from the dashboard and test scripts	JSON answers; live log feed
Protected resource	Stores the highest token seen per key and rejects writes with an older token.	`{key, token, owner, value}`	Accepted or REJECTED stale-token
Dashboard (`public/`)	Draws the ring, role table, locks, fencing demo, live terminal and event log; has the "Break things" buttons.	Gateway JSON every 500 ms	The web page
Tests (`test.js`, `test-chaos.js`, `bench.js`)	Smoke and fault-injection checks, randomised safety test, benchmarks.	A running or self-started cluster	PASS/FAIL lines; `BENCHMARKS.md`
Virtual Lab (`vlab/`)	Six experiments with theory, quizzes and a guided simulation of the cluster in the browser.	Visitor actions	Explanations and simulated cluster

4.4 Data / Inputs / Dataset

The project uses no dataset. Its inputs are API requests and fault commands. A lock request carries a **resource name** (any text), a **client name**, a **lease length** (`ttlMs`, clamped to 500 to 120,000 ms, default 10,000) and, for renew and release, the **fencing token** returned by the grant. Test inputs are the scripted scenarios in `test.js`, the random schedule of crashes, restarts and partitions in `test-chaos.js`, and the client counts in `bench.js`. No personal or confidential data is processed.

Table 4: Gateway API summary

Endpoint (gateway, port 8080)	Method	Purpose
`/api/acquire`, `/api/renew`, `/api/release`	POST	Lock operations, forwarded to the leader
`/api/cluster`, `/api/locks`, `/api/resource`, `/api/term`	GET	Cluster state, held locks, protected resource, live log lines
`/api/resource/write`	POST	Write with a token; rejected if the token is older than the newest seen
`/api/fault/kill`, `/restart`, `/wipe`, `/partition`, `/heal`	POST	Fault injection
`/api/config`, `/api/reset`	GET/POST	Protocol switches (pre-vote, check-quorum, fencing); reset to a clean cluster

4.5 Algorithm / Design Decisions

Leader election

Figure 4 shows `tryElection`. A pre-vote round changes nothing on any disk and decides whether a real election is worthwhile. Only if 3 of 5 machines would vote does the node increase its term, vote for itself, save that to disk and ask for votes. A node grants a vote only if it has not voted for someone else in that term and the candidate's log is at least as up to date as its own; this guarantees that the winner already holds every committed lock.

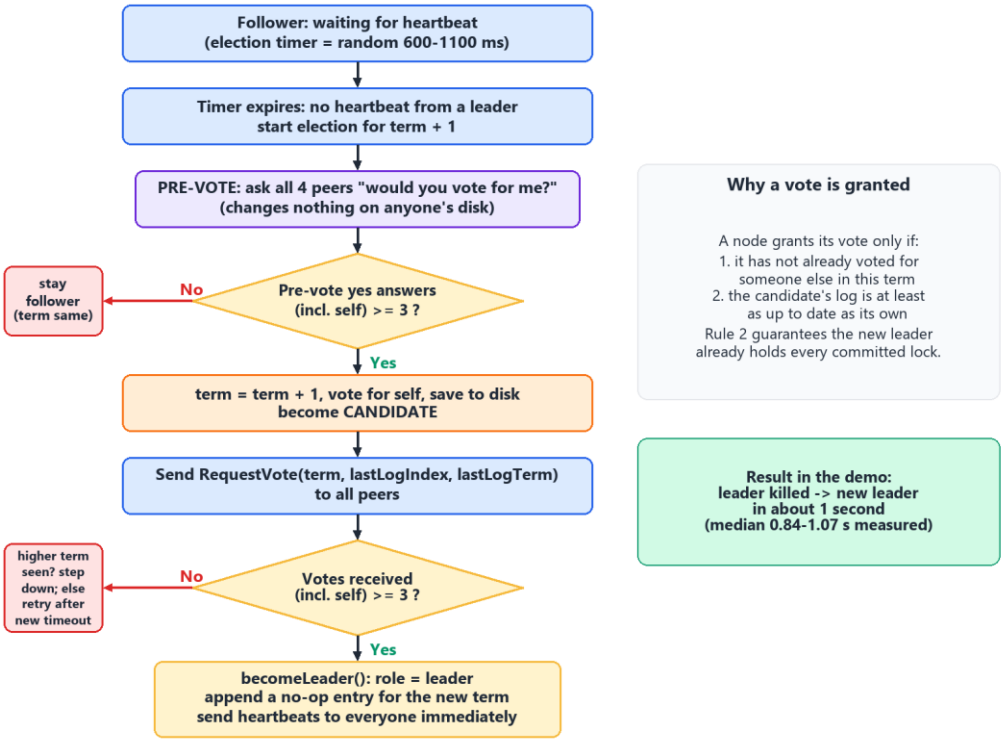


Figure 4: Leader election flow (pre-vote, vote, become leader)

Lock state machine

The lock table is a deterministic function of the committed log. The time used by a command is stored inside the command, not read from each machine's clock, so all five replicas decide identically. The table is not saved: it is rebuilt by replaying the log.

```
case 'acquire': {
  const l = sm.locks[cmd.key];
  if (l && l.expiresAt > cmd.now) {                  // still held
```

```
    if (l.owner === cmd.owner) return { ok: true, token: l.token, reentrant: true };
    return { ok: false, error: 'held', holder: l.owner };
  }
  const token = ++sm.fence;                          // strictly increasing fencing token
  sm.locks[cmd.key] = { owner: cmd.owner, token, expiresAt: cmd.now + cmd.ttlMs };
  return { ok: true, token, expiresAt: cmd.now + cmd.ttlMs };
}
```

Fencing

Figure 5 shows the stale-client case. Client A holds token 2 and freezes. Its lease expires, client B receives token 3 and writes. When A wakes up and writes with token 2, the protected resource rejects it because it has already seen token 3.

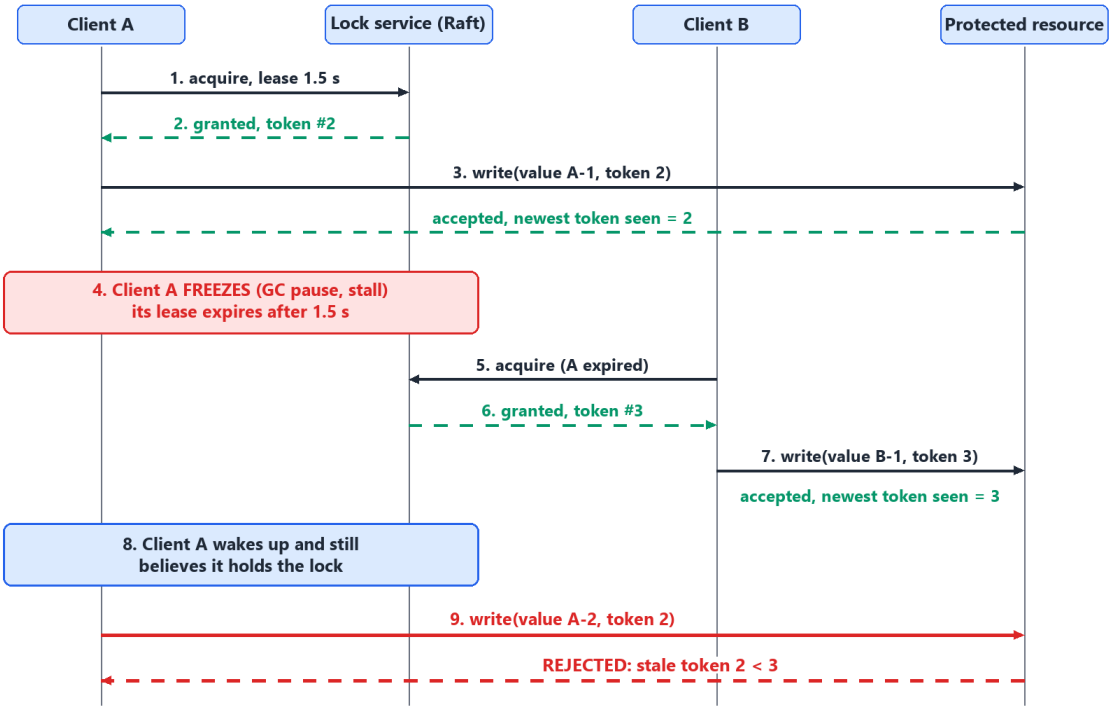


Figure 5: Fencing token rejecting a write from a client whose lease expired

Partitions

When the network splits (Figure 6) only the side with 3 or more machines can elect a leader and commit entries. The old leader on the small side cannot collect acknowledgements, so after 1.5 s it steps down and grants nothing. After the network heals it sees the higher term and copies the entries it missed.

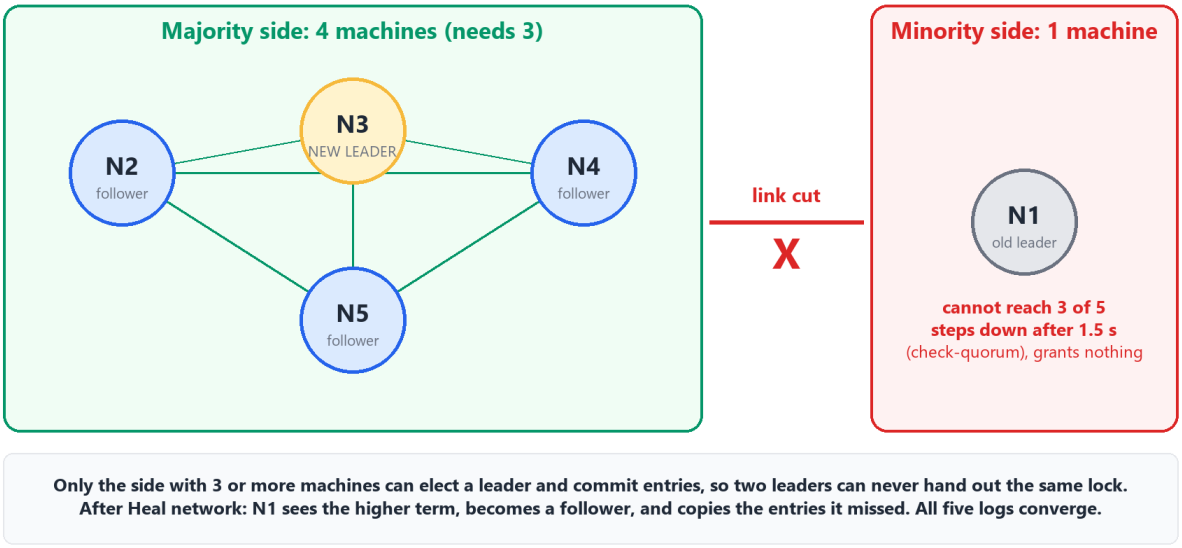


Figure 6: Network partition: the majority side elects a new leader, the cut-off leader steps down

Key design decisions and constants

Table 5: Design decisions

Decision	Reason
Raft messages and client calls over plain HTTP and JSON	Easy to read in logs and to test; no libraries.
Append-only write-ahead log; term and vote saved with write-to-temp-file then atomic rename	A crash cannot leave a half-written term file; a torn last log line is ignored on load.
Lock table rebuilt from the log, not saved	One source of truth; recovery is just replay.
New leader appends a no-op entry	Lets it commit entries from earlier terms safely.
Leader proposes a "sweep" entry to expire leases	Expiry is replicated like any command, so every replica removes the lock at the same point.
Gateway retries acquire and renew but not release	Acquire and renew are safe to repeat; a repeated release could free someone else's lock.
Pre-vote and check-quorum can be switched on or off	Lets the Virtual Lab and tests show what each one prevents.

Table 6: Protocol constants used by the replicas

Constant	Value	Meaning
Heartbeat interval	120 ms	Leader contacts every follower
Election timeout	600 to 1100 ms (random)	Follower waits this long before starting an election
Quorum-loss limit	1500 ms	Leader steps down if no majority has answered
Lease sweep check	300 ms	Leader looks for expired leases
Peer RPC timeout / client operation timeout	300 ms / 3000 ms	Gives up on a silent peer / on an uncommitted request
Majority	floor(N/2) + 1 = 3 for N = 5	Votes or copies needed

5. Methodology and Implementation

5.1 Workflow / Development Method

Work followed five stages. (1) **Requirements and design:** the safety goals (one holder per lock, no lost grants), the failure model, the data flow and the failure strategy were written first. (2) **Implementation:** the replica came first (persistence, then election, then replication and the lock state machine), followed by the gateway and then the dashboard. (3) **Fault injection:** crash, restart, wipe and partition controls were added so that every failure could be triggered from a button. (4) **Evaluation:** scripted checks, a randomised chaos test with a history checker, and benchmarks. (5) **Teaching material:** the Virtual Lab and the demo script. Each stage was checked by running the system before the next began.

5.2 Technologies, Tools and Resources

Table 7: Technologies and tools

Category	Technology / Tool	Purpose / Justification
Programming language / runtime	Node.js 18 or newer (v24.19.0 used for the runs in this report)	Built-in `http`, `fetch`, `fs` and `child_process` are enough, so there are no dependencies [9].
Frontend	HTML, CSS and plain JavaScript	A single static page; no build step.
Storage	Local files: JSON-lines write-ahead log and a small JSON term file	Simple, inspectable durability.
Development / testing tools	VS Code; `test.js`, `test-chaos.js`, `bench.js`; headless Google Chrome for the report screenshots	Editing, automated checks, and repeatable screenshots.
Packaging	Dockerfile and `docker-compose.yml` (provided; not run for this report)	Replicas as containers on a private network.
Hardware / cloud	One Windows 11 laptop (benchmarks recorded on an Intel Core i7-12700H); no cloud	Everything runs locally at no cost.

5.3 Implementation Details

Replica process. `replica.js` (about 440 lines) is started as `node replica.js <id> <clusterSize> <dataDir>`. It serves HTTP for peers, the gateway and the log feed. A 120 ms timer sends heartbeats if the node leads, or starts an election if the election timer ran out.

Persistence. Before replying to any message the replica saves its term and vote (write to a temporary file, then rename) and appends new log entries to the write-ahead log. On boot, `loadDisk` reloads both and the lock table is rebuilt as the log is re-applied. Setting `DURABLE=1` adds an fsync to every write.

Replication and commit. `propose` appends a client command to the leader's log and wakes `replicate` for each follower, which sends up to 64 entries per message. `advanceCommit` counts how many nodes have stored the newest entry of the current term and commits it at 3. Followers that are behind are repaired by backing up `nextIndex` until the logs match; conflicting entries on a follower are removed.

Gateway. `viaLeader` keeps a cached leader, asks all nodes when the cache is empty, follows "not-leader" hints and retries up to five times. `startNode` and `stopNode` spawn and kill replica processes, which is how the Kill and Restart buttons really work. Partitions are injected by telling each replica which peers to ignore.

Dashboard. `public/app.js` polls `/api/cluster` and `/api/locks` every 500 ms and redraws. It contains no lock logic. The live terminal reads `/api/term`, which merges the gateway's own log lines with the log lines of all five replicas in time order.

Challenges and fixes. A replica that stepped down had pending client requests, which now return "leadership lost" instead of hanging. A cut-off leader used to answer lock-table reads, so reads now require a recent majority. The replica file was renamed from `node.js` to `replica.js` so that `npm run` works on Windows. A torn last line in the log after a crash is ignored when loading.

5.4 User Interface / Prototype / Output

The screenshots below were captured from the running system (a clean cluster on ports 8090 and 7101 to 7105) using headless Chrome driven through the DevTools protocol. Each one shows a real state of the running processes.

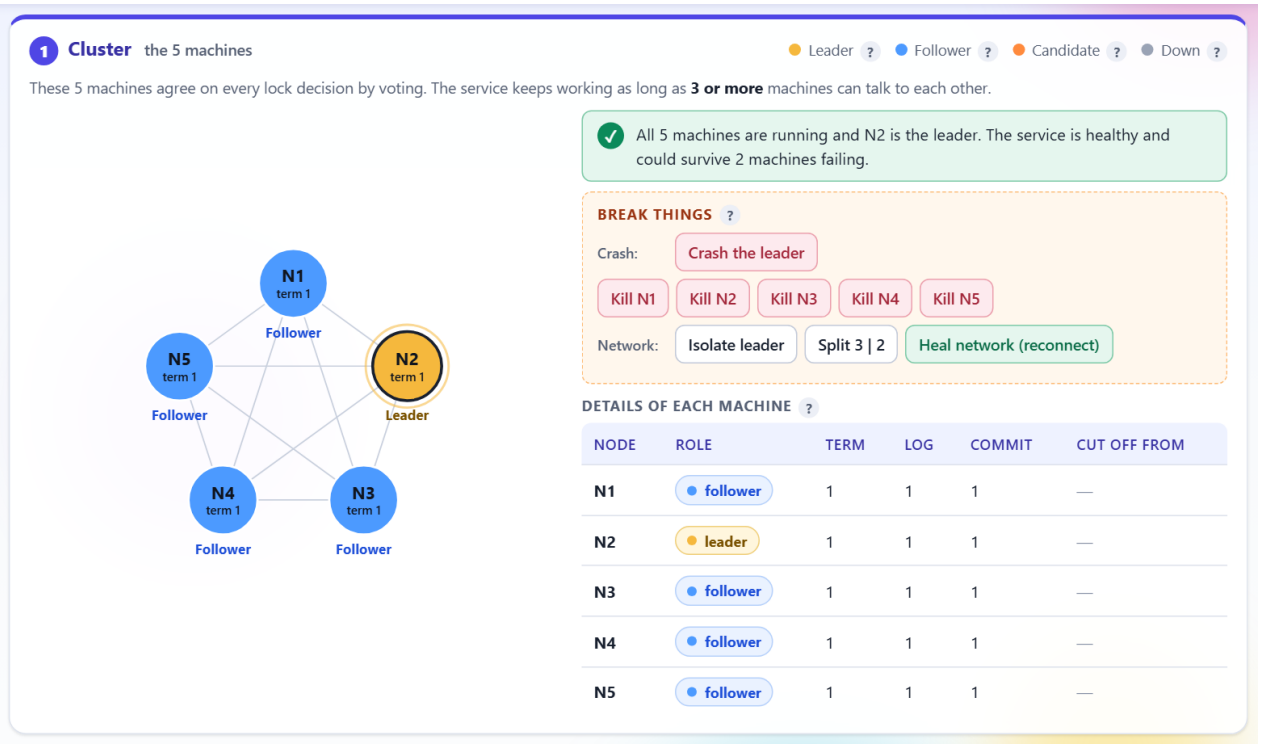


Figure 7: Dashboard, Cluster card: five machines, one leader (gold), same term and commit index, "Break things" controls

Figure 7 shows the healthy cluster: the leader in gold, four blue followers, identical term, log and commit values in the table, and the buttons used to cause failures.

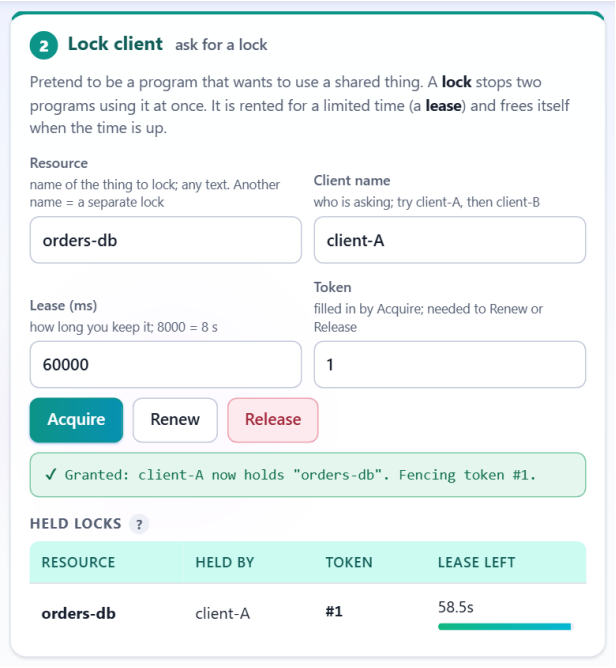


Figure 8: client-A acquires "orders-db" and gets token 1

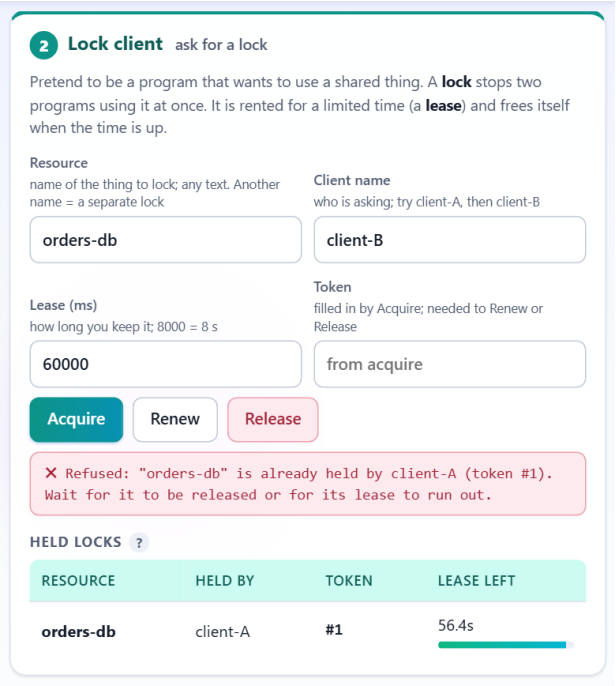


Figure 9: client-B is refused: held by client-A

Figures 8 and 9 demonstrate mutual exclusion. The same resource name is requested by two clients; only the first is granted. The lease bar counts down the remaining lease.

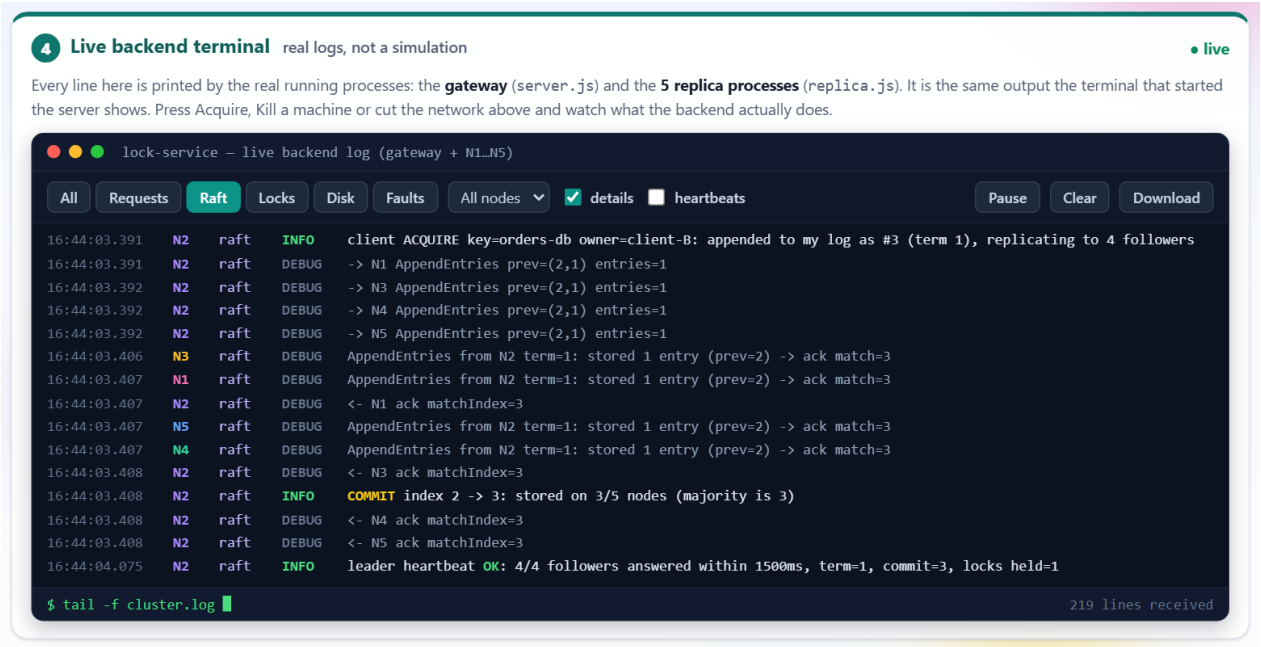


Figure 10: Live backend terminal, Raft filter: AppendEntries to followers and "COMMIT … stored on 3/5 nodes (majority is 3)"

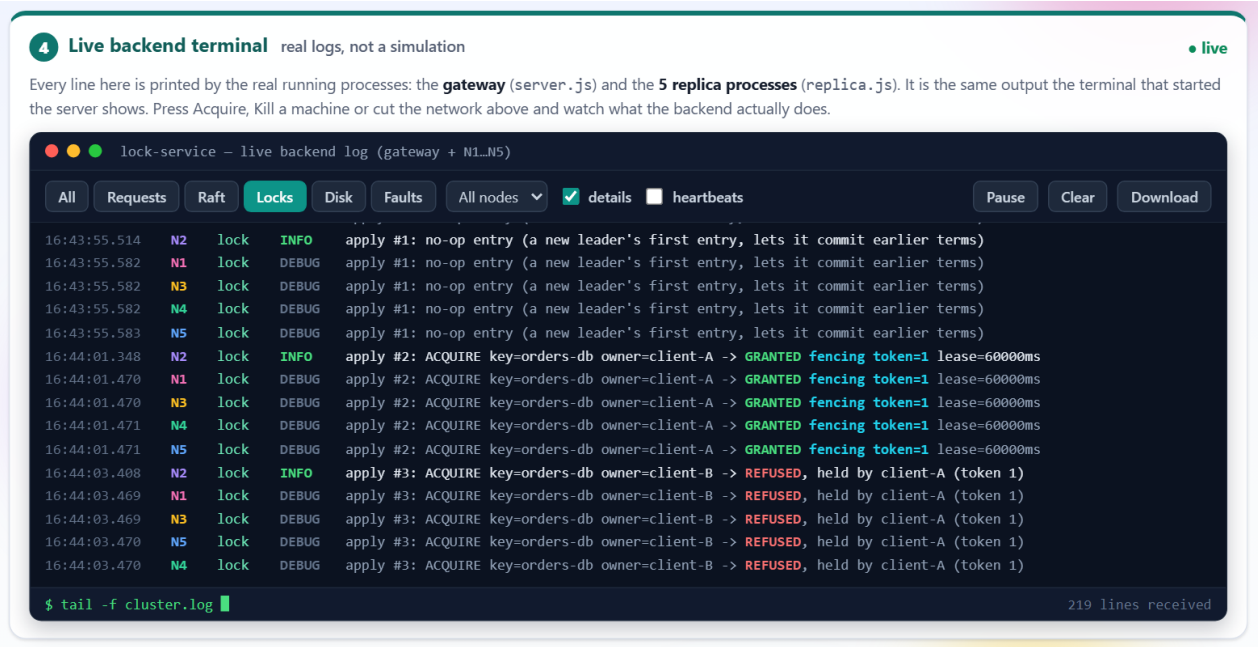


Figure 11: Live backend terminal, Locks filter: GRANTED and REFUSED applied on the replicas

Figures 10 and 11 are the real log lines of the replica processes: the leader replicates, commits at 3 of 5, and the lock decision is applied.

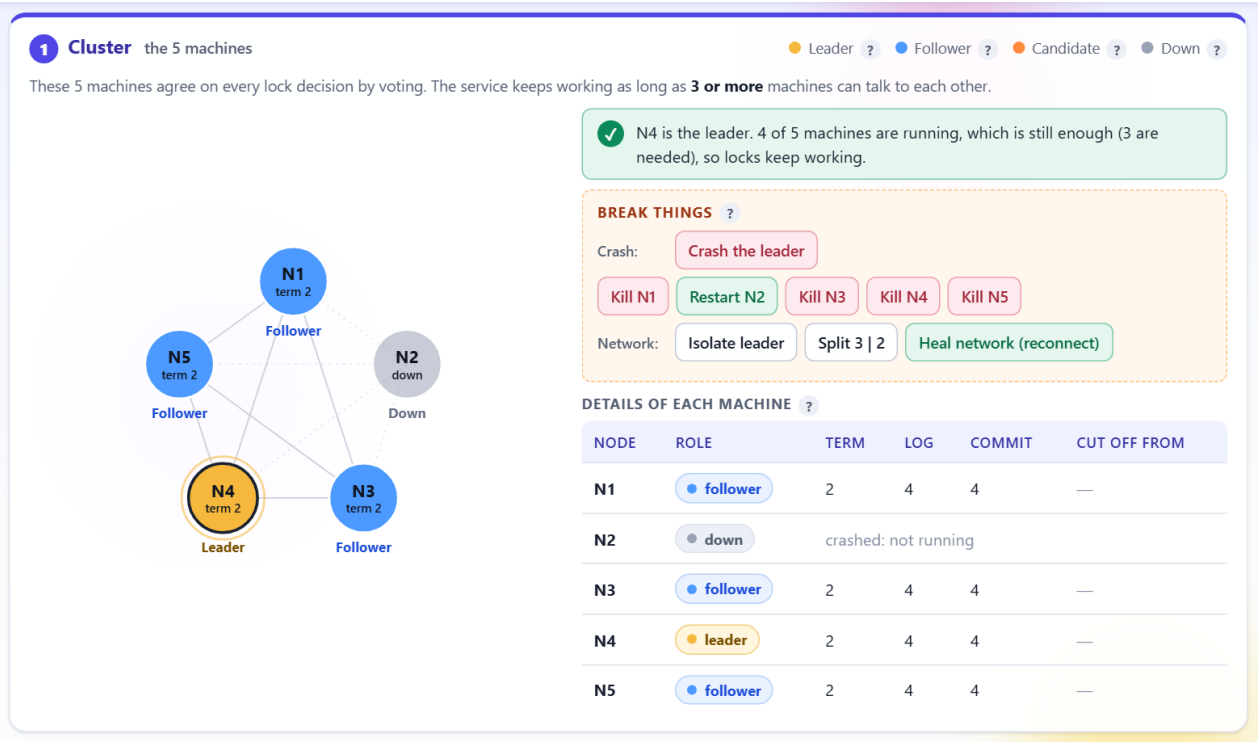


Figure 12: After "Crash the leader": the old leader is down and a new leader has been elected in a higher term

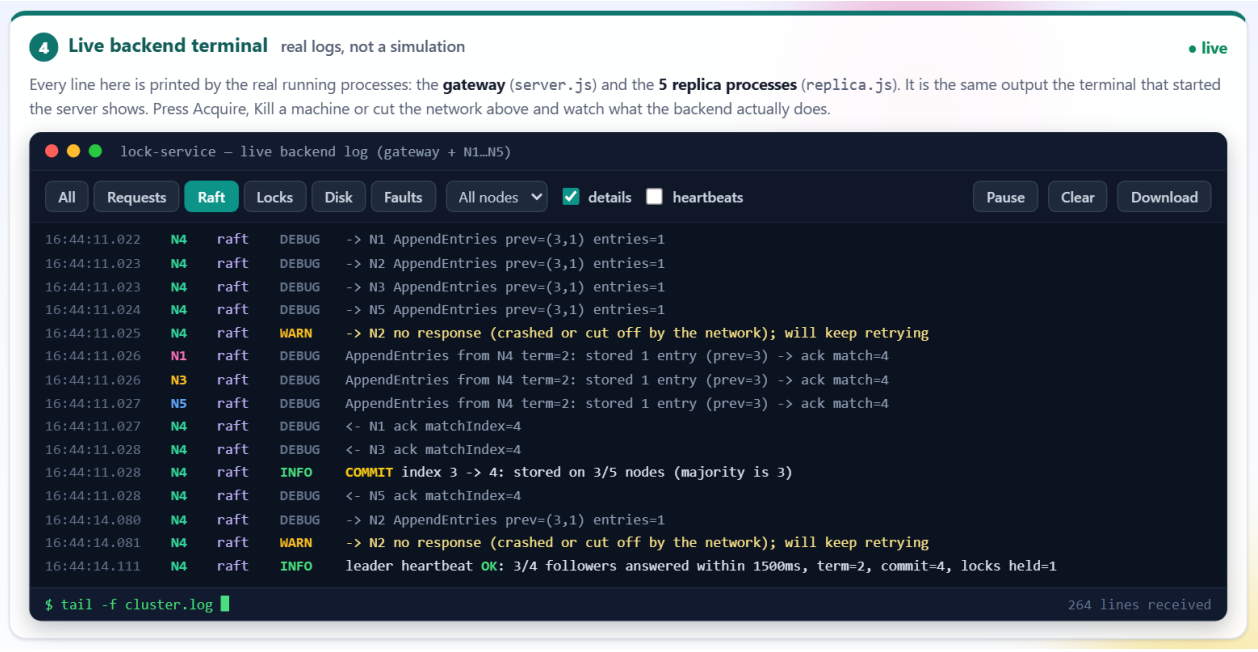


Figure 13: Live terminal during failover: replication to the surviving followers and the new leader's heartbeat

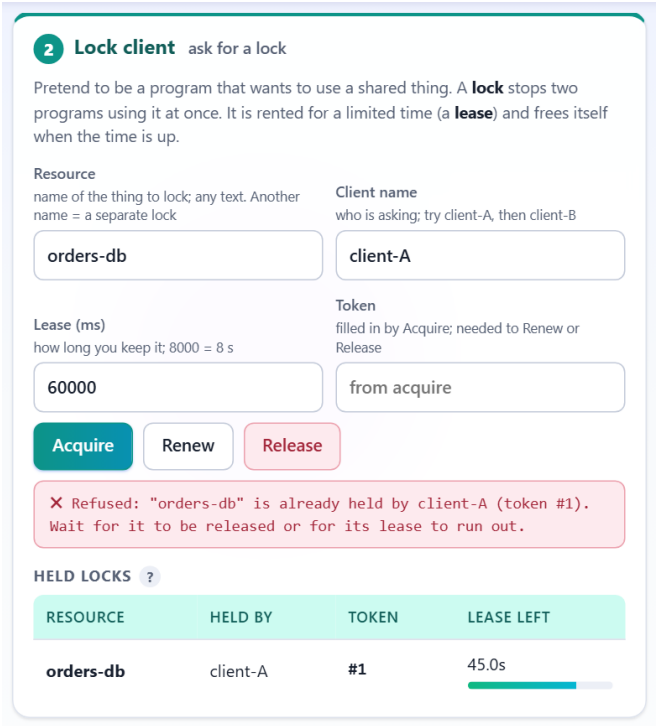


Figure 14: The lock is still held by client-A after the leader crashed

Figures 12 to 14 show failover. The ring shows the new leader, the terminal shows the cluster working with four nodes, and the lock table still holds the lock, because the grant was committed on a majority before the crash.

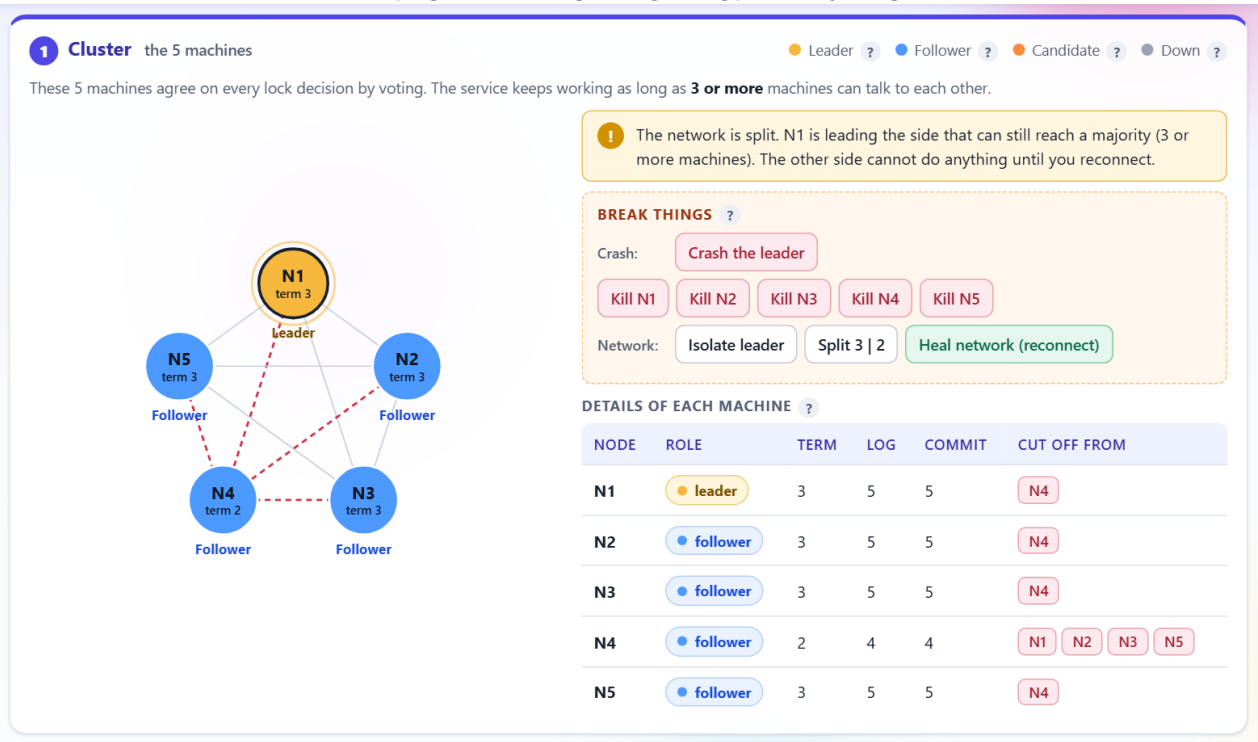


Figure 15: Partition: red dashed links show the cut; the table lists which machines each node cannot reach

Figure 15 shows a partition in progress. The side that kept a majority elected a leader and continues; the isolated node shows an older term and a shorter log until the network is healed.

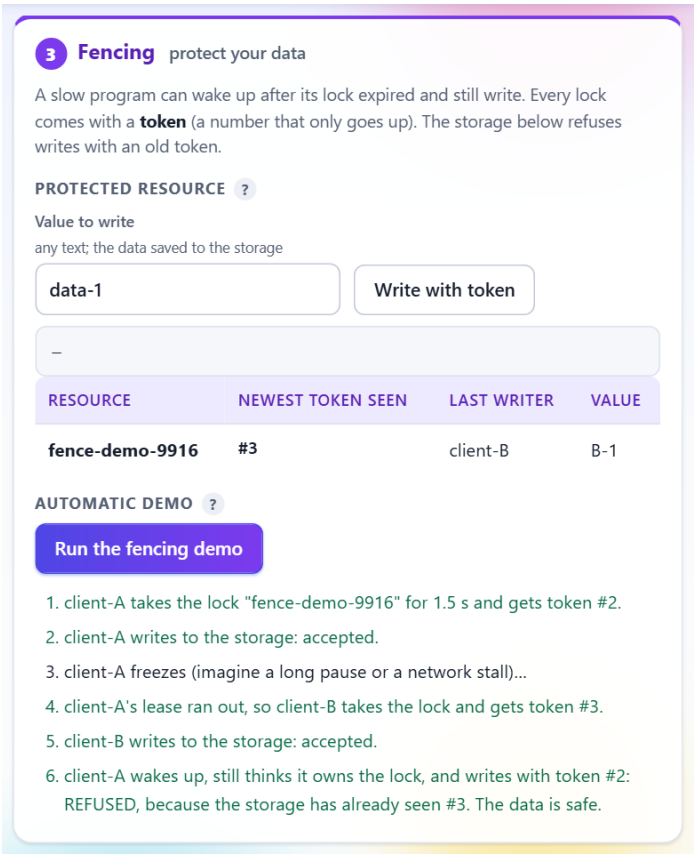


Figure 16: Fencing demo: the write by client-A with the old token is REFUSED because the resource has already seen a newer token

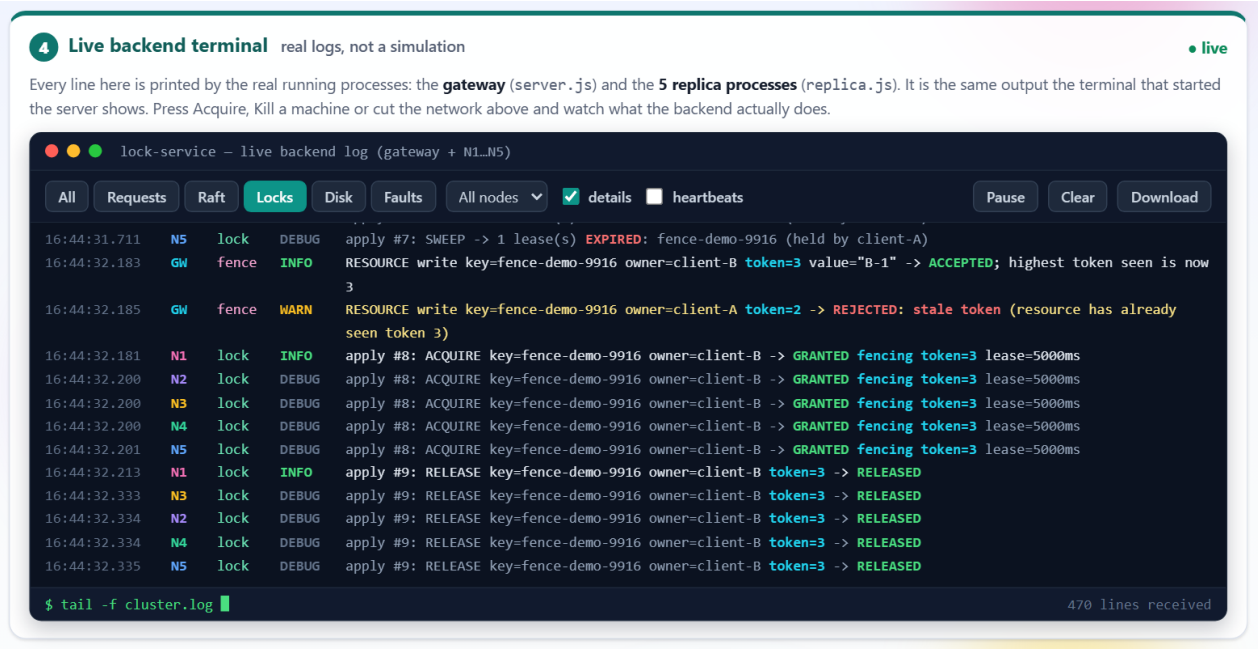


Figure 17: Live terminal: the protected resource logs accepted writes and the rejected stale write

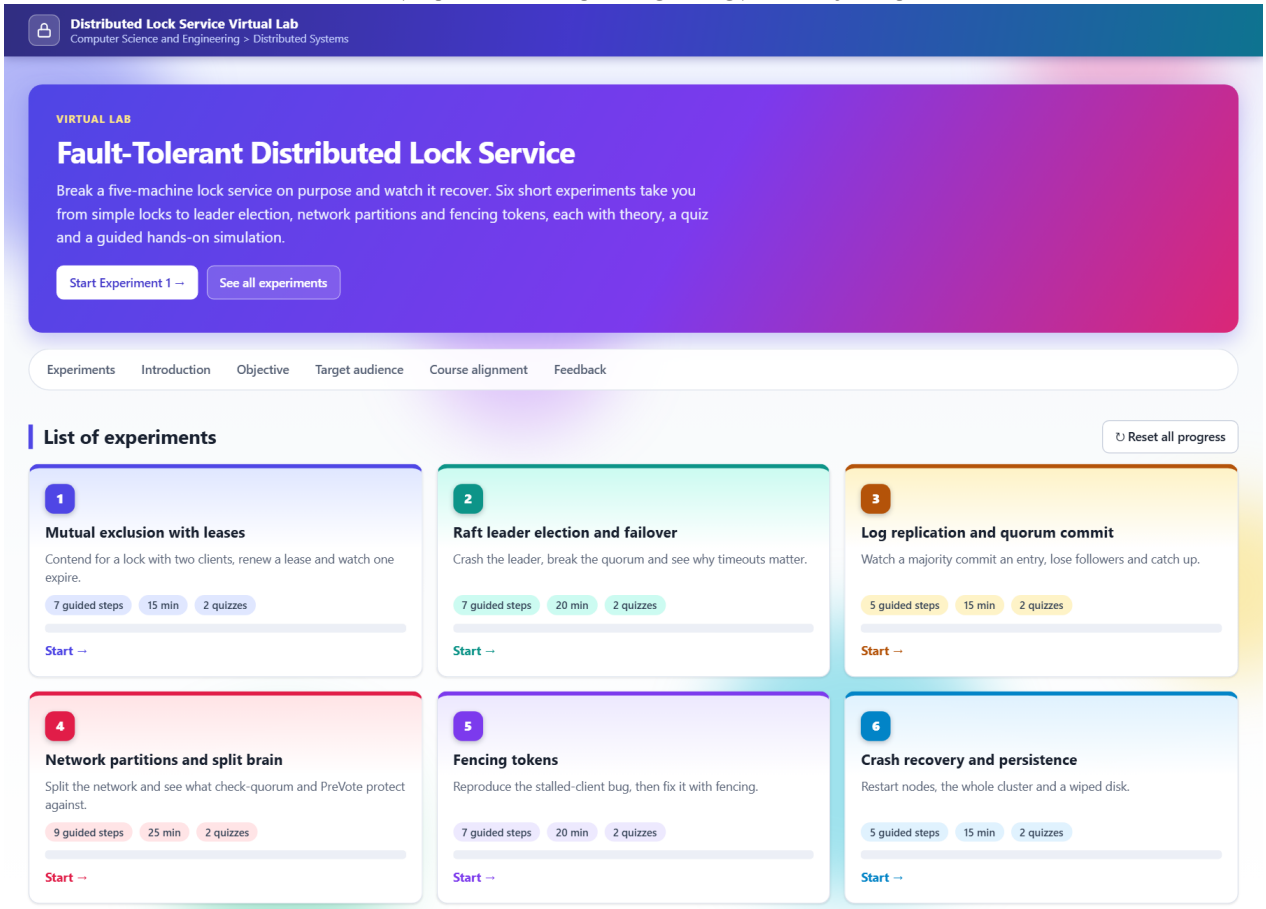


Figure 18: Virtual Lab home page: six experiments

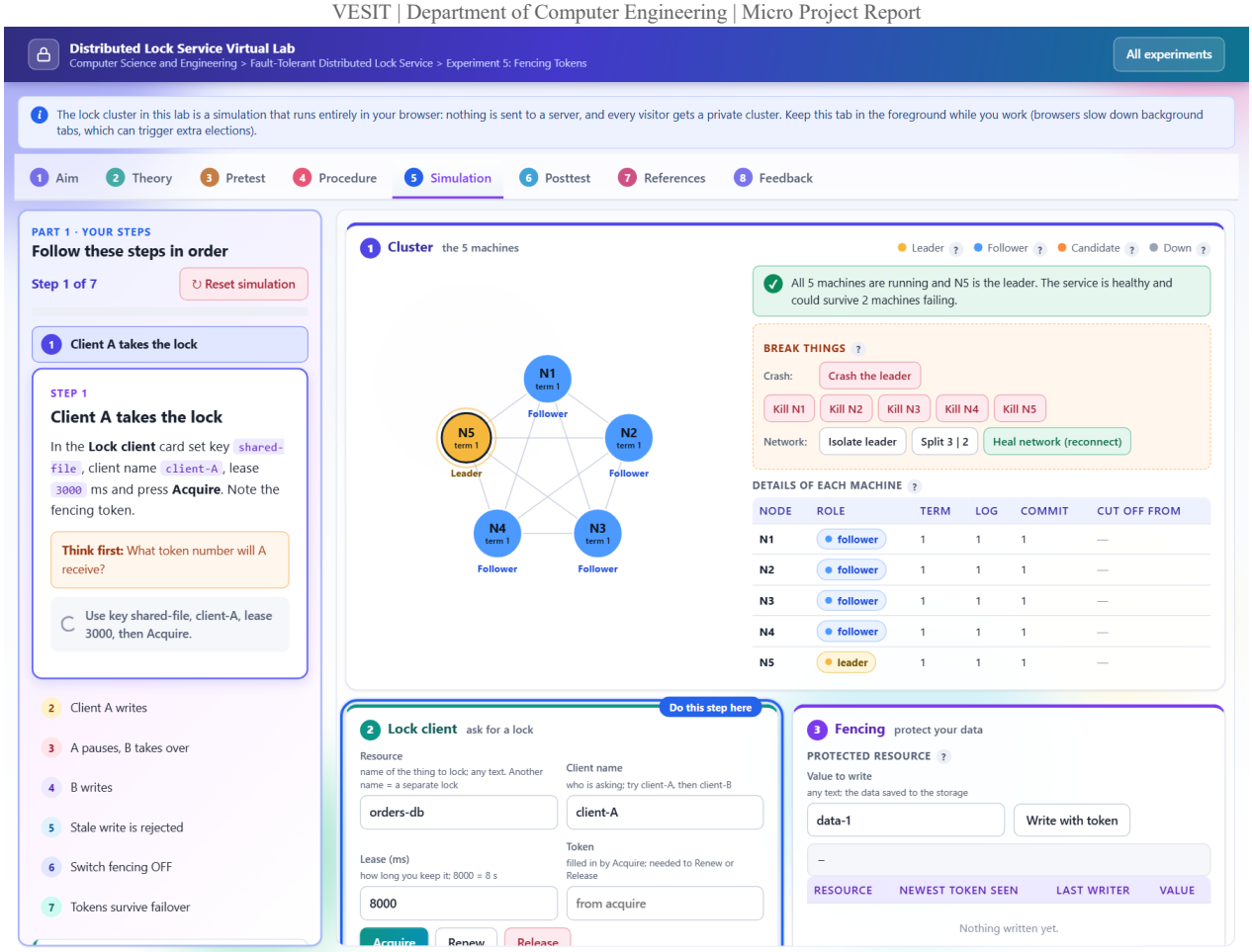


Figure 19: Virtual Lab, Experiment 5 (Fencing Tokens), Simulation tab with guided steps

Figures 18 and 19 show the Virtual Lab. Its cluster is a simulation that runs entirely in the visitor's browser, so nothing is sent to a server.

5.5 Team Contribution

Table 8: Team contribution

Team Member	Assigned Responsibility	Work Completed / Evidence
Tanmay Maniyar	Design, implementation, testing and documentation (add other members here if the project was done in a group)	Replica (Raft and lock state machine), gateway, dashboard, tests, chaos test, benchmarks, Virtual Lab, this report

6. Testing, Evaluation and Results

6.1 Test Plan and Test Cases

Test environment: Windows 11, Node.js v24.19.0, all processes on one laptop. The tests were run on 5 November 2026 against a freshly started cluster (gateway on port 8090, replicas on 7101 to 7105). Expected results come from the safety goals in Section 2. The scripted checks cover normal use (acquire, release), failure cases (leader crash, partition) and invalid use (a second client asking for a held lock, a stale token). Token numbers in the table depend on how many locks the cluster had already granted.

Table 9: Scripted fault-injection test cases (`npm test`, 12 checks)

Test ID	Test Input / Scenario	Expected Result	Observed Result	Status
TC-01	Start the cluster	A leader is elected	N1 elected	Pass
TC-02	client A acquires a lock	Granted with a fencing token	Granted, token 4	Pass
TC-03	client B asks for the same lock	Refused, held by A (mutual exclusion)	Refused (held)	Pass
TC-04	Kill the leader	A different node becomes leader	N5 became leader	Pass
TC-05	Read the lock after failover	A still holds the lock	Lock present, owner A	Pass
TC-06	B asks again after failover	Still refused	Refused	Pass
TC-07	Let the lease expire, B acquires	B is granted the lock	Granted	Pass
TC-08	Compare tokens of successive grants	Strictly increasing	4 < 5 < 6	Pass
TC-09	Isolate the leader from the network	Majority side elects a new leader	N2 elected on the majority side	Pass
TC-10	Acquire during the partition	Served by the majority side	Served by a majority-side node	Pass
TC-11	Heal the network	Exactly one leader	One leader	Pass
TC-12	Read the lock table after healing	Same state everywhere, nothing lost	Lock granted during the partition is present	Pass

```
mini-project  -  npm test (12 checks against the live cluster)

$ npm test

PASS   leader elected  N1
PASS   A acquires  token 4
PASS   B is denied (mutual exclusion)
PASS   new leader after leader crash  N5
PASS   lock survives failover
PASS   still exclusive after failover
PASS   lease expiry lets B acquire
PASS   fencing tokens strictly increase  4 < 5 < 6
PASS   majority side elects new leader under partition  N2
PASS   majority side serves writes
PASS   single leader after heal (no split brain)
PASS   state converged after heal

ALL PASSED
```

Figure 20: Output of the 12-check test script: all checks pass

The randomised chaos test (`test-chaos.js`) starts its own cluster, runs 8 clients that acquire, renew and release 3 locks while crashes, restarts, partitions and leader isolations are injected at random, and then checks the recorded history. The 20-second quick version was run for this report. It injected 11 faults (including killing the leader, killing nodes and a 3 | 2 split).

Table 10: Chaos test checks (`npm run chaos:quick`)

Check	What it proves	Observed	Status
Mutual exclusion	No two grants of the same key overlap in time	116 grants checked, no overlap	Pass
Fencing tokens unique	Each token belongs to one key and is used once	Unique	Pass
Grants durable	Final counters are at least the highest token seen	Max token 116, counters 116 on all 5 nodes	Pass

Check	What it proves	Observed	Status
All nodes up	Crashed nodes recover after healing	All 5 up	Pass
One leader	No split brain after healing	Exactly one leader	Pass
Logs identical	Replication converged and committed	Log length 939 on all 5 nodes	Pass
Counters identical	State machines agree	Identical	Pass
Liveness	A fresh lock can be acquired after the chaos	Acquire succeeded	Pass

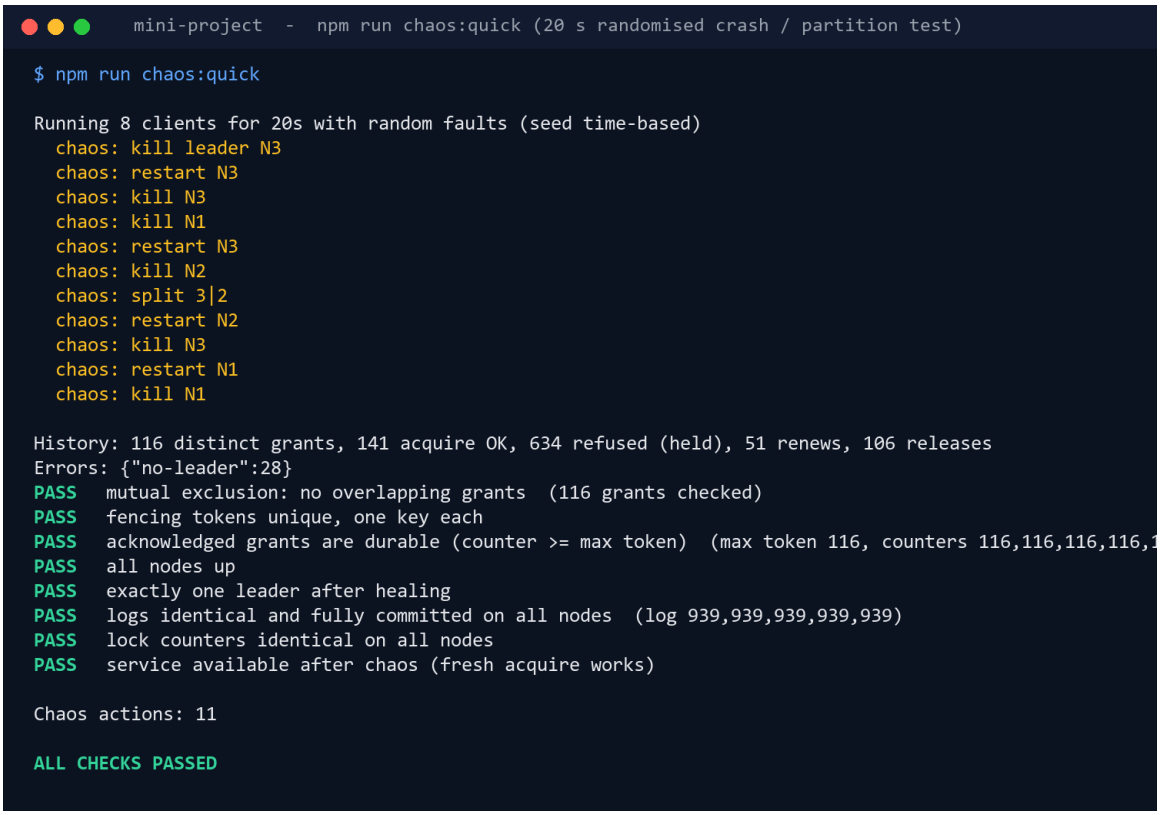


Figure 21: Output of the 20-second chaos test with 11 injected faults

6.2 Evaluation Metrics

Table 11: Evaluation metrics

Metric	Measurement Method / Formula	Observed Value	Interpretation
Operation latency (acquire)	Time from request to committed answer, sequential client, 5 replicas, fsync off	p50 30.6 ms, p95 34.3 ms, p99 45.4 ms	About 31 ms per lock operation; it does not grow with 3, 5 or 7 replicas in these runs
Throughput	Committed operations per second with 1, 8 and 32 concurrent clients (5 replicas, fsync off)	38, 924 and 3,616 ops/s	Throughput rises with concurrency because entries are batched
Throughput with fsync	Same, 5 replicas, fsync on	36, 424 and 219 ops/s	Durable writes cost throughput at high concurrency
Failover time	Leader killed until a client can acquire again; 5 trials	Median 1,073 ms (min 835, max 1,117)	Dominated by the 600 to 1100 ms election timeout
Safety violations	History checker in the chaos test	0 overlapping grants in 116	Mutual exclusion held under random faults
Fault tolerance	Cluster size and majority rule	Survives 2 of 5 crashed machines	Needs 3 machines to commit

The benchmark numbers come from `docs/BENCHMARKS.md`, produced by `node bench.js` on 20 October 2026 on an Intel Core i7-12700H laptop under Windows with Node v24.19.0. They were not re-run for this report, and each configuration was run once, so small differences between rows are within run-to-run variation.

6.3 Results and Discussion

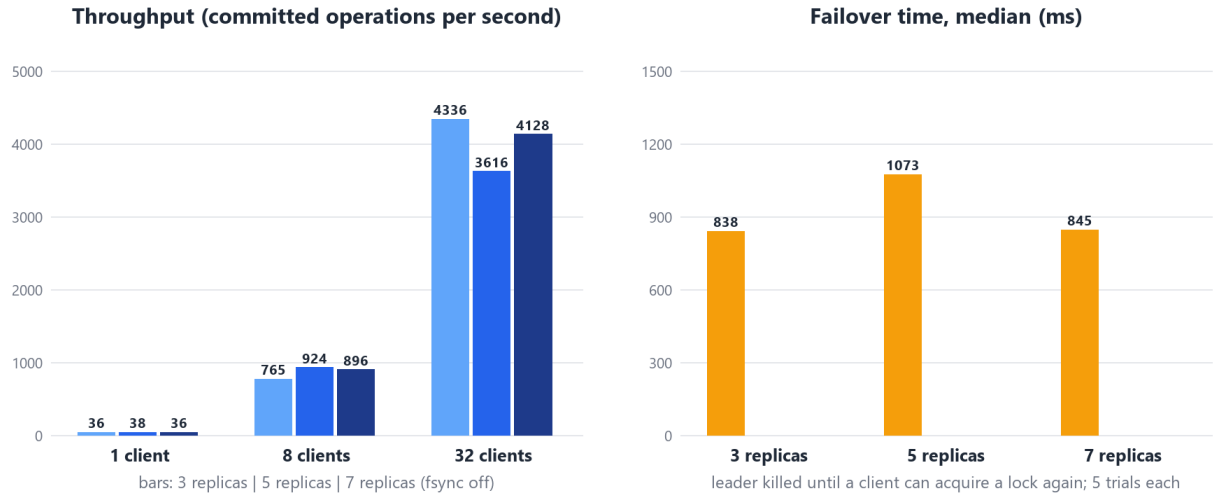


Figure 22: Throughput by cluster size and number of clients, and median failover time

- **Safety held in every run.** All 12 scripted checks and all 8 chaos checks passed, including a run with 11 random faults. No two clients held the same lock at the same time.
- **Failover is about one second.** The median was 0.84 to 1.07 s for 3, 5 and 7 replicas. It is set mainly by the election timeout, not by the cluster size.
- **Latency is flat, throughput depends on concurrency.** A single client sees about 30 ms per operation whatever the cluster size; with 32 clients the cluster commits 3,600 to 4,300 operations per second. The 5-replica run is lower than the 3- and 7-replica runs at 32 clients, which is most likely run-to-run variation because each point was measured once.
- **Durability has a price.** With fsync on, throughput at 32 clients dropped to 219 ops/s, so the default keeps fsync off and offers `DURABLE=1` for stricter durability.
- **Availability is traded for safety.** In the chaos run 28 requests failed with "no-leader" while the cluster had no leader or no majority. These requests were refused, never granted wrongly.

6.4 Validation / Demonstration Evidence

Demo steps: (1) start with `npm run demo` in `mini-project/` and open `http://localhost:8080`; (2) acquire `orders-db` as `client-A`, then as `client-B` and see the refusal; (3) press "Crash the leader" and watch a new leader and the surviving lock; (4) press "Isolate leader", acquire another lock, then "Heal network"; (5) run the fencing demo; (6) run `npm test` in a second terminal. The live terminal shows the real log lines of each process, and the same lines are printed in the terminal that started the server.

Source code, documents and scripts are in the project repository `https://github.com/TanmayManiyar/dist-lock-service`. No passwords, keys or personal data are stored in it or in this report.

7. Sustainability and SDG Alignment

7.1 Sustainability Assessment

Table 12: Sustainability assessment

Dimension	Questions to Consider	Project-Specific Assessment / Evidence
Environmental	Energy and compute use; paper/material use; e-waste; resource efficiency; emissions; lifecycle impacts.	**Neutral to slightly negative, not measured.** The system runs on an existing laptop, with six small Node.js processes and no extra hardware, so no new devices or e-waste are created. Five replicas use about five times the compute of one server in exchange for availability, and timers wake every 120 ms. Energy was not measured (see 7.4).
Social	Accessibility and inclusion; user safety; privacy; fairness; digital divide; community benefit.	**Positive.** The Virtual Lab is a static site that needs no installation or account, so students on modest devices can use it. The dashboard explains terms in plain language, has tooltips, and uses live regions for status messages. No personal data is collected; only lock and client names typed by the user are stored. Safeguard: the admin controls of the gateway can be protected with `BASIC_AUTH`.
Economic	Development and operating cost; maintenance; affordability; productivity; long-term viability.	**Positive.** Development cost was zero (free tools, existing laptop). There are no licences or dependencies to maintain, which lowers long-term cost. Running five real replicas in the cloud would cost money; that cost was not estimated here.

7.2 UN Sustainable Development Goals (SDGs)

Table 13: SDG alignment

SDG No. and Name	Project Feature / Activity Linked to SDG	Expected or Observed Contribution	Evidence / Indicator
SDG 9: Industry, Innovation and Infrastructure (target 9.1, reliable and resilient infrastructure)	Replicated lock service that tolerates crashes and partitions without granting a lock twice.	Observed in tests: a small, open design of the kind used to keep digital infrastructure dependable.	12 of 12 checks and 8 of 8 chaos checks passed; survives 2 of 5 machine failures; failover about 1 s.
SDG 4: Quality Education (target 4.4, relevant technical skills)	Guided dashboard, live backend terminal, demo script and a Virtual Lab with six experiments.	Expected: helps learners see consensus and fencing working. Learner outcomes were not measured.	6 experiments, each with aim, theory, pretest, procedure, simulation, posttest; no installation needed.

Other goals were considered and not claimed. The project has no direct link to SDG 7 or SDG 13: it does not reduce energy use or emissions, and its own energy use was not measured.

7.3 Sustainability Measures and Trade-offs

- **No dependencies and no build step** keep the software small and easy to maintain for years.
- **Bounded memory:** each replica keeps only its last 600 log lines in memory and the gateway keeps 2,500; per-heartbeat lines are off unless the "heartbeats" box is ticked.
- **Data minimisation:** only lock names, client names and tokens are stored.
- **Static hosting for the Virtual Lab:** the simulation runs in the visitor's browser, so no server compute is needed per visitor.
- **Trade-off (availability against resources):** five replicas cost more compute than one but remove the single point of failure; three replicas would tolerate only one failure.
- **Trade-off (durability against speed):** fsync on every write protects against power loss but cut throughput from 3,616 to 219 ops/s at 32 clients.

7.4 Sustainability Indicator / Follow-up

Measured in this project: failover time (about 1 s), operations per second, and the number of safety violations (zero). **Proposed future measures (not measured):** energy used per 1,000 committed operations, idle CPU use of a replica, Virtual Lab page weight and load time, and the share of learners who complete the six experiments.

8. Feasibility, Limitations and Risk Considerations

8.1 Feasibility

Technical: feasible; the full system is about 440 lines for the replica and 240 lines for the gateway, and it ran correctly in all tests. **Operational:** one command starts it; one browser tab operates it. **Cost:** no cost for development, as shown below. A real deployment would need five machines in three availability zones; the deployment notes describe this but it was not built or validated.

Table 14: Cost estimate

Item / Resource	Estimated Cost (₹)	Basis / Notes
Hardware / hosting / cloud	0	Existing laptop; no cloud used. A five-VM cloud deployment is not costed here.
Software / API / dataset	0	Node.js, VS Code and Chrome are free; no dataset; no licensed libraries.
Other resources	0	Virtual Lab can be served by any static host; hosting plan and price not verified.
Total estimated cost	0	One-time development cost only; recurring cost is zero when run locally.

8.2 Limitations

- The log has **no compaction or snapshots**, so it grows without limit.
- The cluster size is **fixed**; machines cannot be added or removed while running.
- Crashes and partitions are **simulated in software on one machine**. The multi-host and Docker set-ups are provided but were not validated for this report.
- Benchmarks use **loopback HTTP** on one laptop, were run once per configuration, and are not cloud numbers.
- The chaos test is randomised and time-limited (20 s here). It finds bugs but is not a formal proof of correctness.
- Leases rely on the **leader's clock** being reasonably accurate.
- The Virtual Lab's simulation is a separate implementation of the same logic, not the real replica code.

8.3 Risks, Ethics, Privacy and Security

Table 15: Risks and safeguards

Risk	Impact	Safeguard / Note
Admin endpoints (`/api/fault/*`, `/api/reset`) have no login by default	Anyone who can reach the gateway could crash nodes or wipe data	Run on localhost; set `BASIC_AUTH`; do not expose the gateway to the internet

Risk	Impact	Safeguard / Note
Traffic and disk files are plain text; no TLS	Lock names and tokens could be read on a shared network	Local demo only; add TLS and file permissions before real use
Clock error on the leader	A lease could end too early or too late	Lease times are stamped once by the leader; keep leases long compared with clock error
Client skips the token check	A stale client could damage data	Resources must verify tokens (demonstrated by the protected resource)
Wrong claims about reliability	Users trust a prototype with real data	Limitations stated; all evidence in this report comes from runs shown here

Privacy and ethics: no personal data or credentials are processed or stored. **Responsible use of AI tools:** AI-assisted tools were used during development and when preparing this report; the code was run and the results, screenshots and test output shown here come from those runs.

9. Conclusion and Future Scope

9.1 Summary

The project built a working fault-tolerant lock service from scratch: five Raft replicas, a deterministic lock state machine with leases and fencing tokens, a gateway with leader discovery and fault injection, a dashboard with a live backend terminal, and a Virtual Lab. All seven objectives were met. The service granted each lock to one client at a time in all runs, kept committed locks through leader crashes and partitions, recovered nodes from disk, and failed over in about one second. The evidence is the 12 passing scripted checks, a chaos run with 116 grants and no overlap, the benchmark tables and the screenshots. The project supports SDG 9 through resilient infrastructure design and SDG 4 through the Virtual Lab; it has no demonstrated effect on energy or emissions.

9.2 Future Scope

- Add **log compaction and snapshots** so the log does not grow forever.
- Support **changing the cluster membership** while running.
- Validate the **Docker and multi-host** deployment on real machines or cloud VMs and measure real network latency.
- Add **TLS, authentication and access control** to the gateway and replicas.
- Provide a **client library** that renews leases in the background and carries tokens automatically.
- Run longer and more varied chaos tests and a formal specification check.
- **Measure energy** per committed operation and learner outcomes for the Virtual Lab.

References

[1] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in Proc. USENIX Annual Technical Conf. (USENIX ATC ’14), Philadelphia, PA, USA, Jun. 2014, pp. 305–319.

[2] M. Burrows, “The Chubby lock service for loosely-coupled distributed systems,” in Proc. 7th USENIX Symp. Operating Systems Design and Implementation (OSDI ’06), Seattle, WA, USA, Nov. 2006, pp. 335–350.

[3] M. Kleppmann, “How to do distributed locking,” Blog post, Feb. 8, 2016. [Online]. Available: https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html

[4] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free coordination for Internet-scale systems,” in Proc. USENIX Annual Technical Conf. (USENIX ATC ’10), Boston, MA, USA, Jun. 2010.

[5] D. Ongaro, “Consensus: Bridging theory and practice,” Ph.D. dissertation, Dept. Comput. Sci., Stanford Univ., Stanford, CA, USA, 2014.

[6] C. Gray and D. Cheriton, “Leases: An efficient fault-tolerant mechanism for distributed file cache consistency,” in Proc. 12th ACM Symp. Operating Systems Principles (SOSP ’89), Litchfield Park, AZ, USA, Dec. 1989, pp. 202–210.

[7] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O’Reilly Media, 2017.

[8] The etcd Authors, “etcd documentation.” [Online]. Available: https://etcd.io/docs/

[9] OpenJS Foundation, “Node.js documentation.” [Online]. Available: https://nodejs.org/docs/latest/api/

Appendices

Appendix A: Additional screenshots / outputs

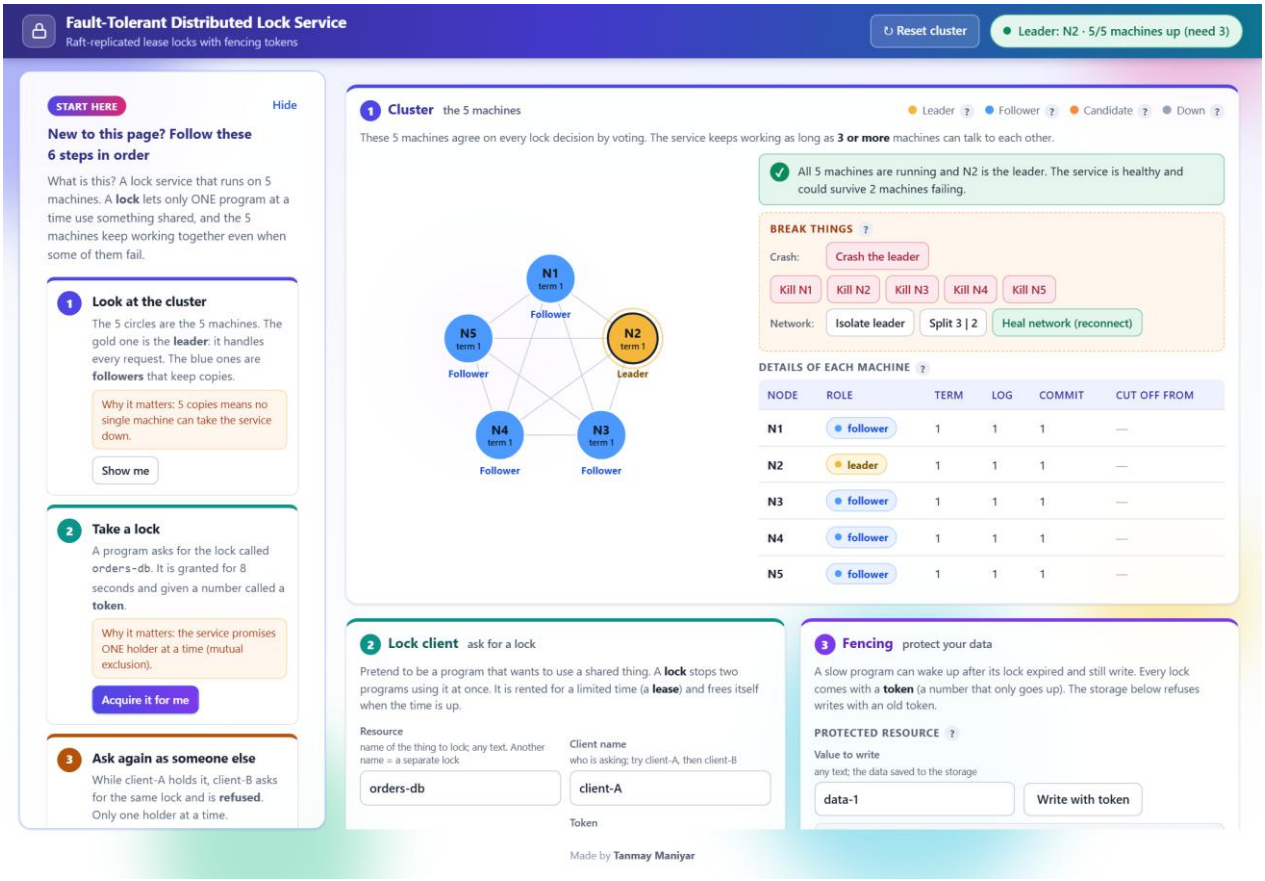


Figure A1: Full dashboard with the "Start here" guide and the three working cards.

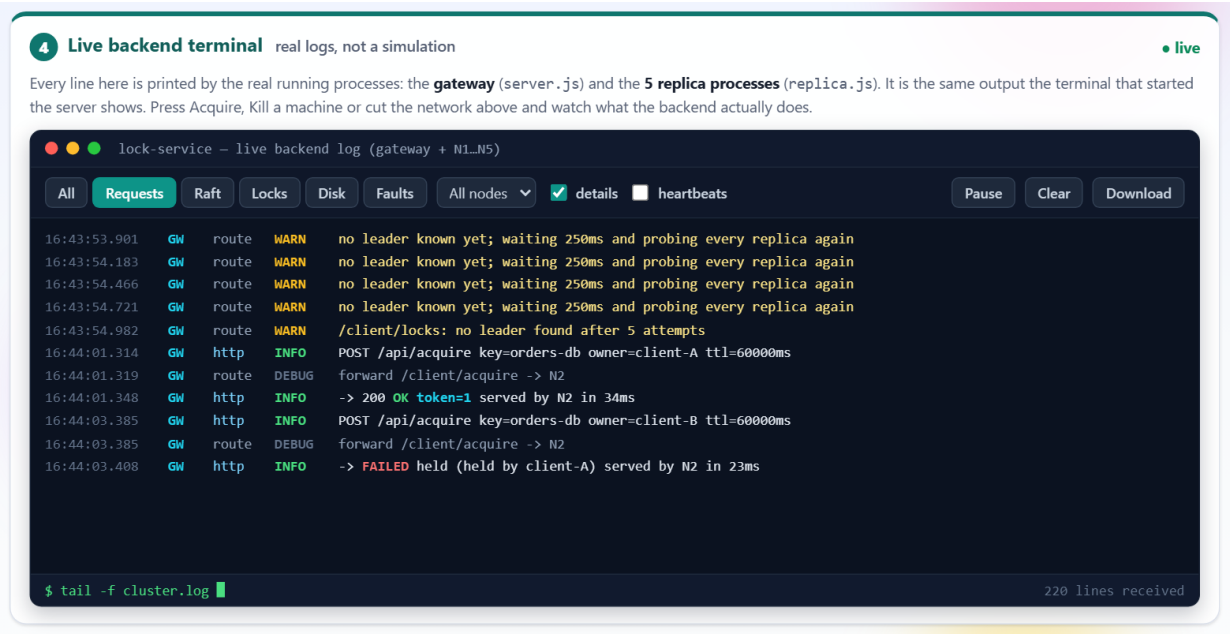


Figure A2: Live terminal, Requests filter: the gateway routes each HTTP request to the leader.

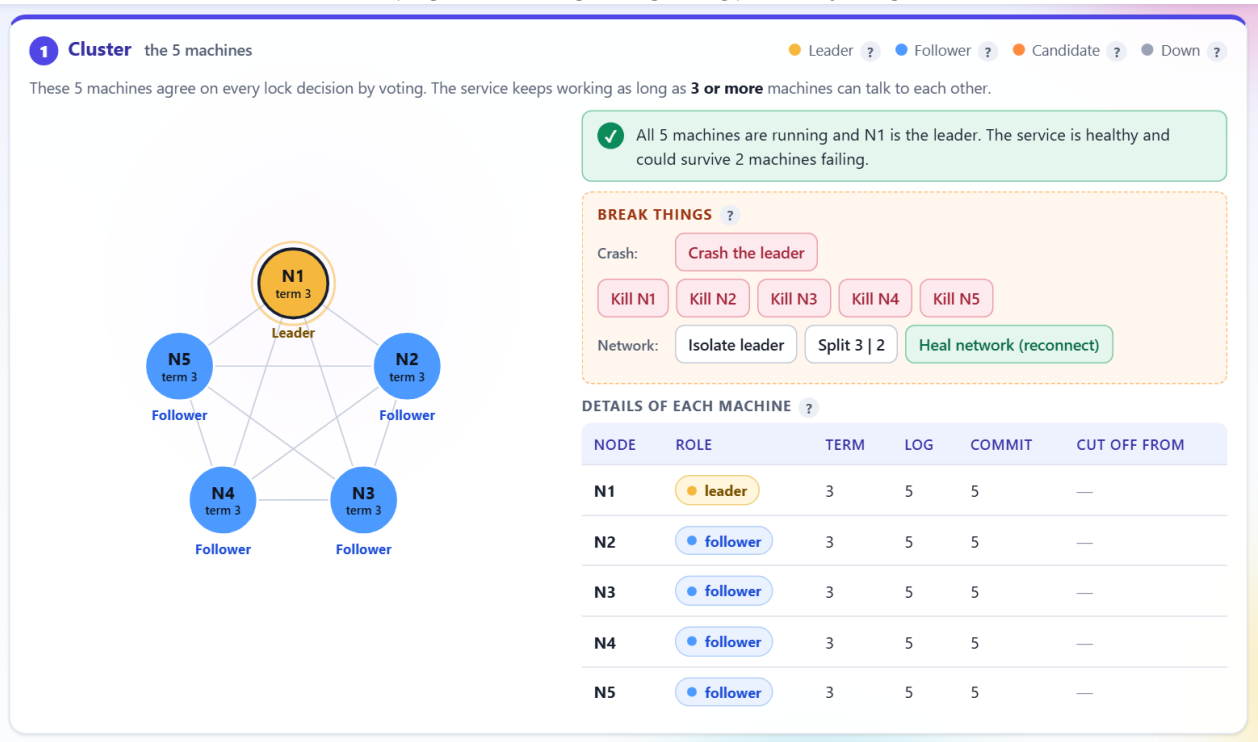


Figure A3: After "Heal network": one leader and the details table with no cut links.



Figure A4: Event log in plain language: faults in red, elections in gold.

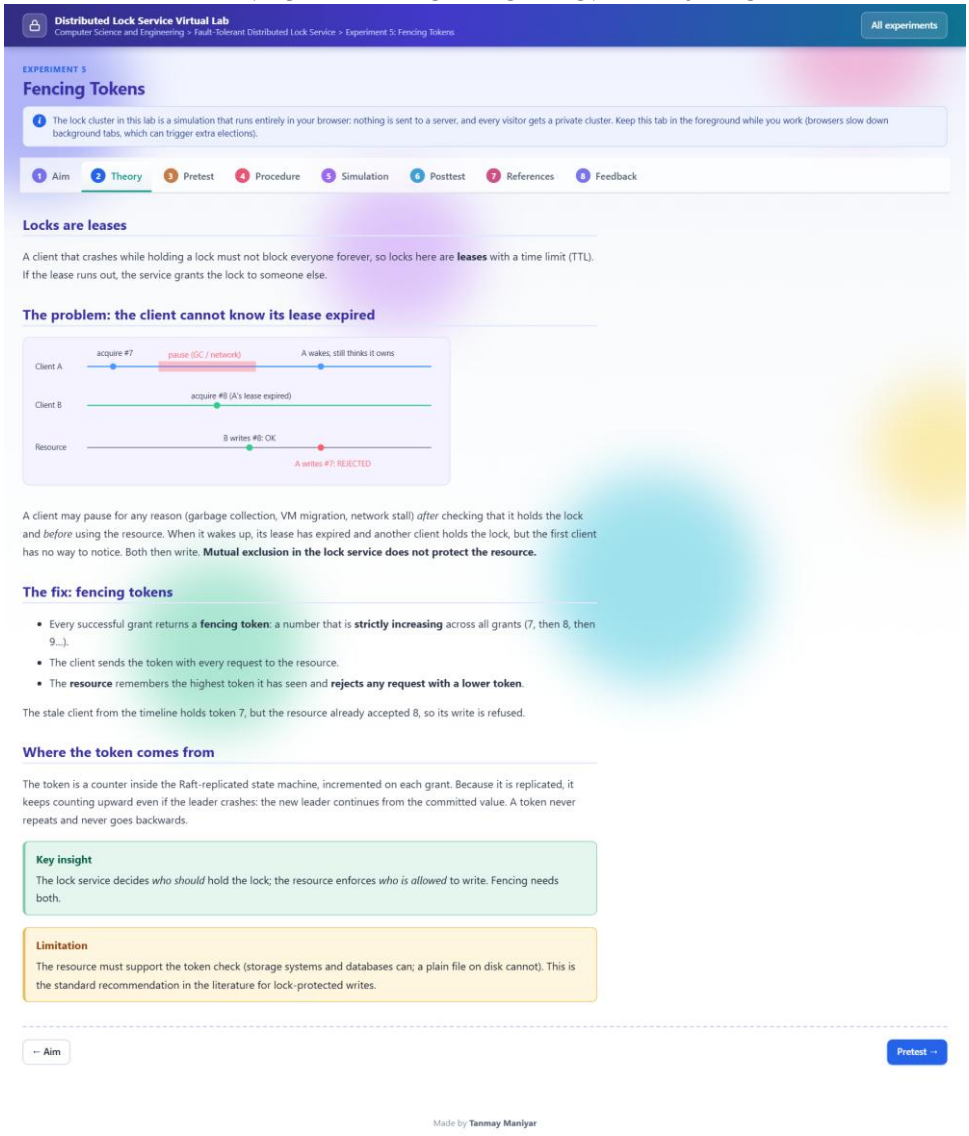


Figure A5: Virtual Lab, Experiment 5 Theory tab: lease timeline and the fencing-token fix.

Appendix B: Selected pseudocode or code excerpts

Majority and commit rule (`replica.js`):

```
const MAJORITY = Math.floor(N / 2) + 1;            // 3 when N = 5

function advanceCommit() {
  for (let n = lastIndex(); n > commitIndex; n--) {
    if (st.log[n].term !== st.term) break;          // only commit entries of the current term
    const count = 1 + peers.filter(p => matchIndex[p] >= n).length;
    if (count >= MAJORITY) { commitIndex = n; applyCommitted(); break; }
  }
}
```

Fencing check in the gateway (`server.js`):

```
const cur = resource[key] || { maxToken: 0 };
if (config.fencing && token < cur.maxToken)
  return json(res, 200, { ok: false, error: 'stale-token', maxToken: cur.maxToken });
resource[key] = { maxToken: Math.max(cur.maxToken, token), value, writer: owner };
```

Vote rule (`replica.js`, simplified):

```
// grant a vote only if: same term, not yet voted for someone else, candidate log at least as up to date
if (b.term === st.term && (st.votedFor === null || st.votedFor === b.from) && upToDate(b.lastLogIndex,
b.lastLogTerm)) {
  st.votedFor = b.from; persist(); grant = true; resetElectionTimer();
}
```

Appendix C: Additional test cases

The 8 chaos checks are in Table 10 and the 12 scripted checks in Table 9. Further runs can be repeated with `npm run chaos` (60 s) and `npm run bench`. The fault-injection test cases of the Virtual Lab are given in its six experiment pages.

Appendix D: Repository / demo / deployment information

- Repository: `https://github.com/TanmayManiyar/dist-lock-service` (folders `mini-project/` and `vlab/`).
- Run the service: `cd mini-project`, then `npm run demo` (opens `http://localhost:8080`); run the tests with `npm test` in a second terminal.
- Run the Virtual Lab: `cd vlab`, then `npm start` (serves `http://localhost:8099`); it can be deployed as static files.
- Ports used: 8080 for the gateway and 7001 to 7005 for the replicas. Environment variables: `PORT`, `NODES`, `BASE_PORT`, `DATA_DIR`, `DURABLE`, `BASIC_AUTH`, `REMOTE_NODES`. No credentials are needed or stored.